

DEUS Dashboard

Deploiement DevOps sur AWS

Michael Rousseau

EPITA — Cycle Ingenieur 2^e annee

Module DEUS

13 juin 2026

Résumé

Ce rapport presente la mise en place d'une infrastructure DevOps complete pour le deploiement d'une application web Angular sur Amazon Web Services. Le projet couvre l'ensemble de la chaine DevOps : conteneurisation avec Docker, infrastructure as code avec Terraform, gestion de configuration avec Ansible, integration et deploiement continus avec GitHub Actions, monitoring avec CloudWatch, et gestion des secrets avec SSM Parameter Store.

Table des matières

1	Introduction	4
1.1	Contexte du projet	4
1.2	Objectifs	4
1.3	Technologies utilisees	5
2	Architecture	6
2.1	Vue d'ensemble	6
2.2	Composants	7
2.2.1	Frontend — S3 Static Website Hosting	7
2.2.2	Backend — ECS (EC2 Launch Type)	7
2.2.3	Bases de donnees	7
2.3	Choix d'architecture	8
2.4	Resilience et scalabilite	8
2.5	Reseau — VPC	9
3	Backend API	10
3.1	Presentation	10
3.2	Structure du projet	10
3.3	Routes API	11
3.4	Dockerfile	11
3.5	Docker Compose — Developpement local	12
4	Terraform — Infrastructure as Code	13
4.1	Presentation	13
4.2	Gestion de l'etat	13
4.3	Organisation en modules	14
4.4	Graphe de dependances	14
4.5	Modules generiques et injection de variables	14
4.6	Variables et secrets	15
4.7	Adaptations pour AWS Learner Lab	15
4.7.1	Exemple — Reference au LabRole	16

5	Ansible — Configuration Management	18
5.1	Presentation	18
5.2	Inventaire	18
5.3	Roles	18
5.3.1	Role <code>security</code>	18
5.3.2	Role <code>docker</code>	19
5.3.3	Role <code>ecs-agent</code>	19
5.3.4	Role <code>cloudwatch-agent</code>	19
5.4	Playbook monitoring	19
5.5	Connexion via Bastion	20
6	CI/CD — GitHub Actions	21
6.1	Presentation	21
6.2	Pipeline CI	21
6.3	Deploiement Frontend	22
6.4	Deploiement Backend	22
6.5	Terraform en CI	22
6.6	Gestion des credentials AWS Learner Lab	22
7	Monitoring et Logging	24
7.1	Presentation	24
7.2	CloudWatch	24
7.2.1	Logs	24
7.2.2	Alarmes	24
7.3	Prometheus	25
7.3.1	Metriques applicatives	25
7.4	Grafana	25
7.4.1	Sources de donnees	26
7.4.2	Dashboard Backend Overview	26
7.4.3	Acces au monitoring	26
7.5	SonarQube — Qualite du code	27
7.5.1	Integration CI	27
7.5.2	Developpement local	28
7.6	Architecture du monitoring	28
8	Securite	29
8.1	Gestion des secrets — SSM Parameter Store	29
8.2	IAM — Adaptations Learner Lab	29
8.2.1	Architecture cible vs. realite Learner Lab	29
8.2.2	Contrainte Learner Lab	30

8.3	Reseau — Segmentation VPC	30
8.4	Bastion SSH	31
8.5	Cle API NASA	31
9	Procedure de deploiement	32
9.1	Prerequis	32
9.2	Etape 1 — Initialisation des credentials	32
9.3	Etape 2 — Bootstrap Terraform	32
9.4	Etape 3 — Deploiement de l'infrastructure	33
9.5	Etape 4 — Configuration Ansible	33
9.6	Etape 5 — Premier deploiement applicatif	33
9.7	Etape 6 — Verification	34
9.8	Ressources deployees	35
9.9	Gestion des sessions Learner Lab	35
10	Conclusion	37
10.1	Bilan technique	37
10.2	Difficultes rencontrees	37
10.3	Ameliorations possibles	38

Chapitre 1

Introduction

1.1 Contexte du projet

Le projet DEUS Dashboard est une application web permettant de visualiser des données astronomiques fournies par les APIs de la NASA. L'application affiche l'image astronomique du jour (APOD), les événements naturels terrestres (EONET), ainsi qu'une carte interactive de la Lune.

L'objectif de ce projet est de mettre en œuvre une méthodologie DevOps complète pour déployer cette application sur Amazon Web Services (AWS), en démontrant la maîtrise des outils et pratiques essentiels du domaine.

1.2 Objectifs

- Conteneuriser l'application avec Docker
- Provisionner l'infrastructure AWS avec Terraform (Infrastructure as Code)
- Configurer les serveurs avec Ansible (Configuration Management)
- Automatiser les déploiements avec GitHub Actions (CI/CD)
- Mettre en place un monitoring avec CloudWatch
- Sécuriser l'infrastructure (IAM, VPC, SSM Parameter Store)

1.3 Technologies utilisees

Categorie	Technologies
Frontend	Angular 20, TypeScript, Leaflet
Backend	Node.js, Express 5, TypeScript
Base de donnees	PostgreSQL 16 (RDS), DynamoDB
Conteneurisation	Docker, Docker Compose
Infrastructure	Terraform, AWS (VPC, ECS, ALB, S3, CloudFront)
Configuration	Ansible
CI/CD	GitHub Actions
Monitoring	CloudWatch (Logs, Metriques, Alarmes, Dashboard)
Securite	IAM, Security Groups, SSM Parameter Store

TABLE 1.1 – Stack technologique du projet

Chapitre 2

Architecture

2.1 Vue d'ensemble

L'architecture suit un modele cloud-native avec separation entre le frontend statique et le backend API.

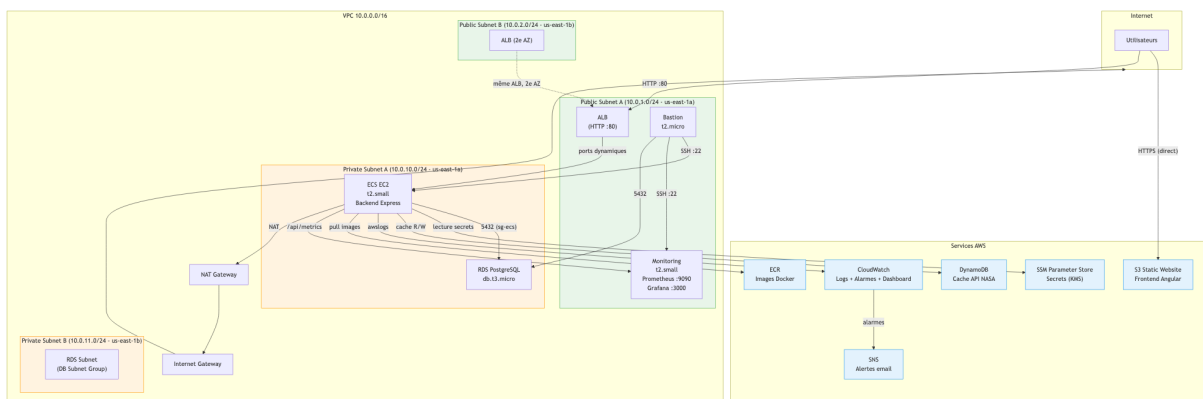
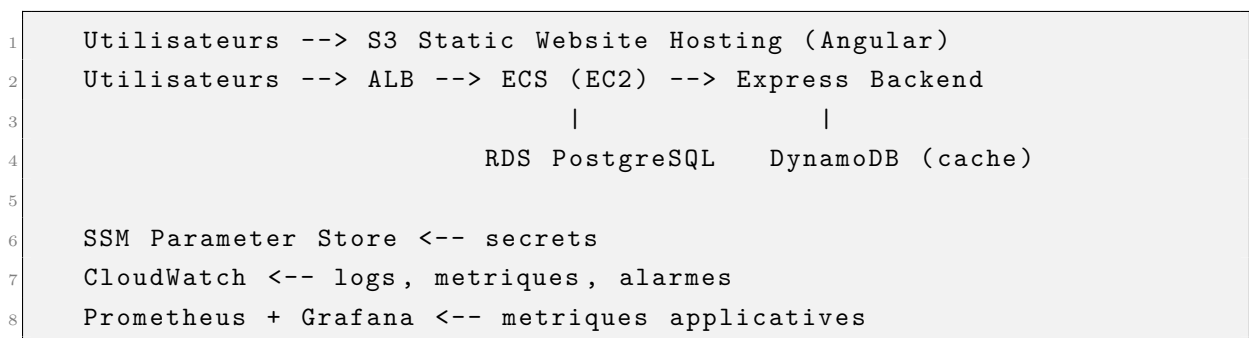


FIGURE 2.1 – Diagramme d'architecture globale



Listing 2.1 – Architecture simplifiée

URLs de deployment :

Frontend deus-dashboard-frontend-dev.s3-website-us-east-1.amazonaws.com

Backend (ALB) deus-dashboard-alb-1071352964.us-east-1.elb.amazonaws.com

Monitoring (Grafana) <http://34.207.159.92:3000>

2.2 Composants

2.2.1 Frontend — S3 Static Website Hosting

L'application Angular est compilee en fichiers statiques et deployee sur le bucket S3 `deus-dashboard-frontend-dev`. Le bucket est configure en mode *Static Website Hosting* avec :

- ACLs publiques pour permettre l'accès en lecture
- Document d'index `index.html`
- Gestion du routage SPA (document d'erreur redirige vers `index.html`)

Note : L'architecture initiale prevoyait CloudFront avec Origin Access Control (OAC) comme CDN. Cependant, l'environnement AWS Learner Lab ne permet pas la creation de distributions CloudFront avec OAC (permissions IAM insuffisantes). La solution adoptee est l'hebergement statique S3 direct, qui reste fonctionnel pour un projet pedagogique.

2.2.2 Backend — ECS (EC2 Launch Type)

Le backend Express est conteneurise et deploye sur ECS avec le type de lancement EC2. Ce choix, plutot que Fargate, permet de :

- Configurer les instances EC2 avec Ansible
- Demontrer la gestion de l'Auto Scaling Group
- Installer des agents de monitoring personnalises

Un Application Load Balancer (ALB) distribue le trafic vers les conteneurs via le port mapping dynamique.

2.2.3 Bases de donnees

RDS PostgreSQL Base relationnelle pour les favoris utilisateur et l'historique de recherche. Instance `db.t3.micro` en single-AZ.

DynamoDB Cache des reponses API NASA avec TTL automatique. Mode de facturation `PAY_PER_REQUEST`.

2.3 Choix d'architecture

Decision	Justification	Alternative
ECS EC2 vs Fargate	Permet Ansible, plus pédagogique	Fargate
Bridge networking	Standard EC2, port mapping dynamique	awsvpc
Single NAT Gateway	Economie (~32\$/mois)	HA NAT
S3 Website vs CloudFront	CloudFront OAC bloqué par Learner Lab	CloudFront
Single-AZ RDS	Projet scolaire, économie	Multi-AZ
Rolling deploy 100/200	Zero-downtime : la nouvelle tâche démarre avant l'arrêt de l'ancienne	Recreate

TABLE 2.1 – Décisions d'architecture et justifications

2.4 Resilience et scalabilité

- **Multi-AZ** : sous-réseaux publics et privés répartis sur `us-east-1a` et `us-east-1b` ; l'ALB est déployé sur les deux AZ et l'ASG peut placer des instances dans chacune (stratégie de placement ECS `spread` sur les AZ).
- **Auto Scaling Group** : 1 à 2 instances ECS. En cas de panne d'instance, l'ASG la remplace automatiquement et l'agent ECS re-enregistre les tâches.
- **Déploiements sans interruption** : le service ECS est configuré avec `deployment_minimum_health_check_timeout` = 100 et `deployment_maximum_percent` = 200. Grâce au port mapping dynamique (mode bridge), la nouvelle tâche démarre sur la même instance, passe le health check ALB, puis l'ancienne est drainée.
- **Auto-reparation applicative** : health checks à deux niveaux — ALB (`/api/health`) qui retire les cibles défaillantes, et `HEALTHCHECK` Docker dans l'image backend.
- **Compromis assumés** : NAT Gateway unique et RDS single-AZ (cf. tableau des décisions) — ces points de défaillance sont acceptés pour limiter les coûts du Learner Lab ; le passage en production consisterait à basculer `rds_multi_az` = `true` et à ajouter un second NAT, sans modification des modules.

2.5 Réseau — VPC

Sous-reseau	CIDR	AZ	Usage
public_a	10.0.1.0/24	us-east-1a	ALB, Bastion
public_b	10.0.2.0/24	us-east-1b	ALB (2 ^e AZ)
private_a	10.0.10.0/24	us-east-1a	ECS, RDS
private_b	10.0.11.0/24	us-east-1b	Groupe RDS

TABLE 2.2 – Allocation des sous-reseaux

Chapitre 3

Backend API

3.1 Présentation

Le backend est une API REST construite avec Express 5 et TypeScript. Son rôle principal est de servir de proxy sécurisé vers les APIs NASA, en cachant la clé API côté serveur.

3.2 Structure du projet

```
1 backend/  
2   src/  
3     index.ts           # Point d'entrée  
4     app.ts             # Configuration Express  
5     config/  
6       index.ts         # Variables d'environnement  
7       database.ts       # Pool PostgreSQL  
8       dynamodb.ts      # Client DynamoDB  
9     routes/  
10      health.ts         # GET /api/health  
11      apod.ts           # GET /api/apod, /range, /random  
12      events.ts         # GET /api/events  
13      favorites.ts       # CRUD /api/favorites  
14     services/  
15      nasaApi.service.ts # Appels HTTP vers NASA  
16      cache.service.ts   # Cache DynamoDB  
17     middleware/  
18      errorHandler.ts
```

Listing 3.1 – Arborescence du backend

3.3 Routes API

Methode	Route	Description	Cache
GET	/api/health	Verification de sante	Non
GET	/api/apod	Image du jour	DynamoDB
GET	/api/apod/range	Images par plage de dates	DynamoDB
GET	/api/apod/random	Images aleatoires	Non
GET	/api/events	Evenements naturels	DynamoDB
GET	/api/favorites	Liste des favoris	Non
POST	/api/favorites	Ajouter un favori	Non
DELETE	/api/favorites/ :date	Supprimer un favori	Non

TABLE 3.1 – Endpoints de l'API

3.4 Dockerfile

Le Dockerfile utilise un build multi-stage pour minimiser la taille de l'image :

```

1 FROM node:20-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci
5 COPY tsconfig.json ./
6 COPY src/ ./src/
7 RUN npm run build
8
9 FROM node:20-alpine
10 WORKDIR /app
11 RUN addgroup -g 1001 appgroup && \
12     adduser -u 1001 -G appgroup -s /bin/sh -D appuser
13 COPY --from=builder /app/dist ./dist
14 COPY --from=builder /app/package*.json ./
15 RUN npm ci --omit=dev
16 USER appuser
17 EXPOSE 3000
18 CMD ["node", "dist/index.js"]

```

Listing 3.2 – Dockerfile multi-stage

Points clés :

- Utilisateur non-root (**appuser**) pour la securite
- **npm ci --omit=dev** en production pour reduire la taille
- Base **node:20-alpine** pour une image legere (~180 Mo)

3.5 Docker Compose — Développement local

```
1 services:
2   backend:      # Express API (port 3000)
3   postgres:     # PostgreSQL 16 (port 5432)
4   dynamodb-local: # DynamoDB Local (port 8000)
```

Listing 3.3 – Services docker-compose

Le fichier `docker-compose.yml` à la racine du projet permet de lancer l'ensemble de la stack localement avec `docker compose up`.

Chapitre 4

Terraform — Infrastructure as Code

4.1 Présentation

Terraform est utilise pour provisionner l'ensemble de l'infrastructure AWS de maniere declarative et reproductible. La configuration est organisee en modules reutilisables.

4.2 Gestion de l'etat

L'etat Terraform est stocke dans un bucket S3 avec verrouillage DynamoDB :

```
1 terraform {  
2   backend "s3" {  
3     bucket      = "deus-dashboard-tfstate"  
4     key         = "dev/terraform.tfstate"  
5     region      = "us-east-1"  
6     dynamodb_table = "deus-dashboard-tf-locks"  
7     encrypt     = true  
8   }  
9 }
```

Listing 4.1 – Configuration du backend S3

Ce mecanisme garantit :

- Le versionnement de l'etat (historique des changements)
- Le verrouillage concurrent (un seul `apply` a la fois)
- Le chiffrement au repos

4.3 Organisation en modules

Module	Ressources
vpc	VPC, sous-reseaux, IGW, NAT Gateway, tables de routage
s3-frontend	Bucket S3, hébergement statique, ACLs publiques
ecr	Repository ECR, politique de cycle de vie
ecs	Cluster ECS, Launch Template, ASG, ALB, Task Definition, Service
rds	Instance PostgreSQL, sous-groupe DB, Security Group
dynamodb	Table de cache avec TTL
bastion	Instance EC2, Security Group SSH
iam	Utilisation de LabRole et LabInstanceProfile pre-existants
ssm	Parametres SecureString (cle NASA, credentials DB)
monitoring	Log Groups, Alarmes CloudWatch, SNS, Dashboard
monitoring-stack	Instance EC2 dediee Prometheus/Grafana, Security Group

TABLE 4.1 – Modules Terraform et leurs ressources

4.4 Graphe de dependances

Les modules sont lies par leurs outputs/inputs :

```

1 VPC (base)
2   |-- Bastion, ECR, S3, DynamoDB (parallele)
3   |-- IAM (LabRole pre-existant, reference par data source)
4   |-- RDS (apres VPC, Bastion)
5   |-- SSM (apres RDS, DynamoDB)
6   |-- ECS (apres tout)
7   |-- Monitoring (apres ECS, RDS)

```

Listing 4.2 – Ordre de deploiement

4.5 Modules generiques et injection de variables

Les modules de `terraform/modules` sont *agnostiques de l'environnement* : aucune valeur specifique (type d'instance, CIDR, version de moteur, seuils d'alarme, adresse e-mail...) n'y est codee en dur. Chaque parametre est une variable documentee, avec une valeur par default generique lorsque c'est sans risque, ou *sans* valeur par default lorsqu'une injection explicite est preferable (ex. `db_name`, dont le changement detruirait l'instance RDS).

La couche environnement (`terraform/environments/dev`) compose les modules et injecte *toutes* les valeurs :

- valeurs non sensibles dans `terraform.tfvars` (versionne);
- secrets via variables d'environnement `TF_VAR_db_password` et `TF_VAR_nasa_api_key` (jamais commites).

Cette separation permettrait de creer un environnement `prod` en dupliquant uniquement la couche environnement (autre `tfvars` : `rds_multi_az = true`, instances plus grandes, CIDR distincts), sans toucher aux modules.

4.6 Variables et secrets

Les valeurs sensibles sont passees via `TF_VAR_*` en CI et ne sont jamais commitees dans le code :

Variable	Type	Source
<code>db_password</code>	sensitive	GitHub Secret
<code>nasa_api_key</code>	sensitive	GitHub Secret
<code>ssh_key_name</code>	string	GitHub Variable

TABLE 4.2 – Variables sensibles Terraform

4.7 Adaptations pour AWS Learner Lab

L'environnement AWS Academy Learner Lab impose plusieurs contraintes qui ont necessite des adaptations de la configuration Terraform :

Contrainte	Probleme	Solution adoptee
Creation de roles IAM	<code>iam:CreateRole</code> non autorise	Utilisation des roles pre-existants <code>LabRole</code> et <code>LabInstanceProfile</code> via <code>data</code> sources Terraform
CloudFront OAC	Impossible de creer des distributions CloudFront avec OAC	Hebergement statique S3 avec ACLs publiques (<code>aws_s3_bucket_public_access_block</code> desactive)
PostgreSQL 16.4	Version non disponible dans le catalogue RDS Learner Lab	Utilisation de PostgreSQL 16.14 (<code>engine_version = "16.14"</code>)
Expiration des credentials	Les cles AWS expirent toutes les ~4 heures	Script de rotation <code>scripts/update-aws-keys.sh</code>
Federation OIDC	Impossible de configurer un provider OIDC pour GitHub Actions	Cles statiques stockees dans les GitHub Secrets, renouvelees manuellement

TABLE 4.3 – Adaptations pour l’environnement Learner Lab

4.7.1 Exemple — Reference au LabRole

Au lieu de creer des roles IAM personnelles, la configuration utilise des `data` sources pour référencer les roles pre-existants :

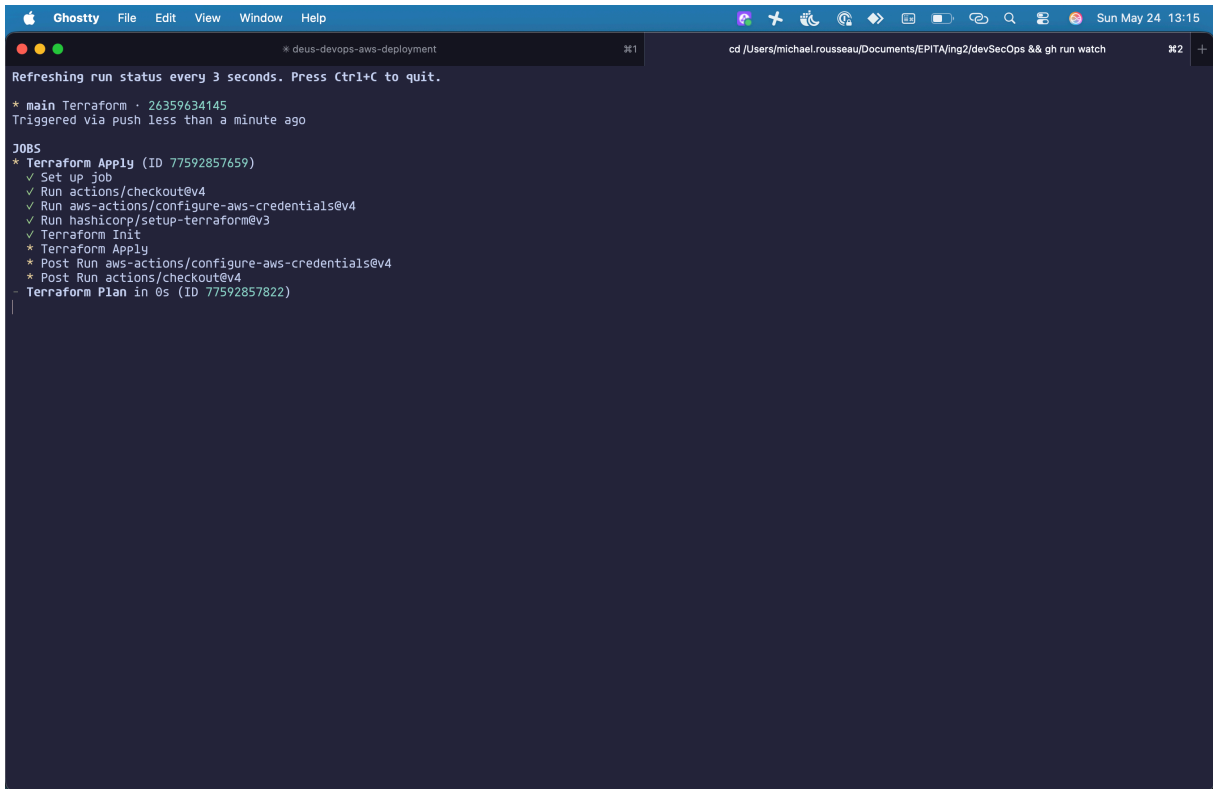
```

1 data "aws_iam_role" "lab_role" {
2   name = "LabRole"
3 }
4
5 data "aws_iam_instance_profile" "lab_profile" {
6   name = "LabInstanceProfile"
7 }
8
9 # Utilisation dans la Task Definition ECS
10 resource "aws_ecs_task_definition" "backend" {
11   execution_role_arn = data.aws_iam_role.lab_role.arn
12   task_role_arn      = data.aws_iam_role.lab_role.arn
13   # ...
14 }
```

Listing 4.3 – Reference au LabRole dans Terraform

Cette approche evite les erreurs `AccessDenied` tout en conservant les permissions

nécessaires (le **LabRole** dispose de permissions suffisantes pour ECS, ECR, SSM, Cloud-Watch et DynamoDB).



```
Ghostty  File  Edit  View  Window  Help
* deus-devops-aws-deployment  #1  cd /Users/michael.rousseau/Documents/EPITA/ing2/devSecOps && gh run watch  #2 +

Refreshing run status every 3 seconds. Press Ctrl+C to quit.

* main Terraform · 26359634145
Triggered via push less than a minute ago

JOBS
* Terraform Apply (ID 77592857659)
  ✓ Set up job
  ✓ Run actions/checkout@v4
  ✓ Run aws-actions/configure-aws-credentials@v4
  ✓ Run hashicorp/setup-terraform@v3
  ✓ Terraform Init
  * Terraform Apply
  * Post Run aws-actions/configure-aws-credentials@v4
  * Post Run actions/checkout@v4
- Terraform Plan in 0s (ID 77592857822)
```

FIGURE 4.1 – Execution de `terraform apply` sur l'environnement dev

Chapitre 5

Ansible — Configuration Management

5.1 Présentation

Ansible est utilise pour configurer les instances EC2 qui hebergent les conteneurs ECS. Il intervient apres le provisionnement Terraform pour installer et configurer :

- Docker et le daemon ECS
- L’agent CloudWatch pour les metriques systeme
- Le durcissement de securite des instances

5.2 Inventaire

L’inventaire est statique (`inventory/hosts.ini`) et regroupe les hotes en trois groupes : `bastion`, `ecs_instances` et `monitoring`. Les adresses IP changent a chaque session Learner Lab (les instances sont recreees ou redemarrees) ; l’inventaire est donc mis a jour a partir des outputs Terraform (`bastion_public_ip`, IP privees des instances ECS et `monitoring`).

Les variables communes aux instances ECS (nom du cluster, region, log group) sont centralisees dans `inventory/group_vars/ecs_instances.yml` et doivent rester coherentes avec les outputs Terraform.

5.3 Roles

5.3.1 Role security

Durcissement des instances :

- Desactivation du login SSH root

- Désactivation de l'authentification par mot de passe
- Installation des mises à jour de sécurité automatiques
- Restriction des permissions cron

5.3.2 Role docker

Installation et activation de Docker CE sur les instances Amazon Linux 2023.

5.3.3 Role ecs-agent

Configuration de l'agent ECS via le template `/etc/ecs/ecs.config` qui spécifie le cluster cible et active les métadonnées de conteneur.

5.3.4 Role cloudwatch-agent

Installation de l'agent CloudWatch unifiée avec collecte :

- Métriques CPU, mémoire, disque
- Logs de l'agent ECS (`/var/log/ecs/ecs-agent.log`)

5.4 Playbook monitoring

Un second playbook (`playbooks/monitoring.yml`) configure l'instance dédiée Prometheus/Grafana. Deux principes de sécurité y sont appliqués :

- **Pas de secret en dur** : le mot de passe administrateur Grafana et le DNS de l'ALB (cible de scraping Prometheus) sont exigés en *extra-vars* — une assertion en `pre_tasks` fait échouer le playbook s'ils sont absents :

```
1 ansible-playbook playbooks/monitoring.yml \
2   -e "alb_dns_name=$(terraform -chdir=../terraform/environments/dev
3     \
4     output -raw alb_dns_name)" \
   -e "grafana_admin_password=$GRAFANA_ADMIN_PASSWORD"
```

Listing 5.1 – Invocation du playbook monitoring

- **Versions épinglées** : les images Docker (`prom/prometheus:v2.53.4`, `grafana/grafana:11.6.1`, `prom/node-exporter:v1.8.2`) sont définies dans les `defaults` de chaque rôle plutôt qu'en `latest`, garantissant des déploiements reproductibles. Un changement de version détecté à l'exécution recrée le conteneur concerné.

5.5 Connexion via Bastion

Les instances ECS sont dans des sous-reseaux priv es. Ansible s’y connecte via SSH ProxyJump a travers le bastion :

```
1 ansible_ssh_common_args: >
2   -o ProxyJump=ec2-user@<bastion_ip>
```

Listing 5.2 – Connexion SSH via Bastion

```

* deus-devops-aws-deployment
TASK [cloudwatch-agent : Install CloudWatch agent] *****
changed: [10.0.10.74]

TASK [cloudwatch-agent : Deploy CloudWatch agent config] *****
changed: [10.0.10.74]

TASK [cloudwatch-agent : Start and enable CloudWatch agent] *****
changed: [10.0.10.74]

RUNNING HANDLER [ecs-agent : Restart ECS agent] *****
changed: [10.0.10.74]

RUNNING HANDLER [cloudwatch-agent : Restart CloudWatch agent] *****
changed: [10.0.10.74]

PLAY RECAP *****
10.0.10.74 : ok=16 changed=6 unreachable=0 failed=0 skipped=1 rescued=0 ignored=0

1m3s → ansible git:(main) ✕ cd /Users/michael.rousseau/Documents/EPITA/ing2/devSecOps/ansible && ansible-playbook playbooks/monitoring.yml -i inventory/hosts.ini

PLAY [Configure monitoring instance (Prometheus + Grafana)] *****

TASK [Gathering Facts] *****
[WARNING]: Host '10.0.1.249' is using the discovered Python interpreter at '/usr/bin/python3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [10.0.1.249]

TASK [docker : Install Docker] *****
ok: [10.0.1.249]

TASK [docker : Start and enable Docker] *****
ok: [10.0.1.249]

TASK [docker : Add ec2-user to docker group] *****
ok: [10.0.1.249]

TASK [node-exporter : Check if Node Exporter is running] *****
ok: [10.0.1.249]

TASK [node-exporter : Run Node Exporter container] *****
skipping: [10.0.1.249]

TASK [prometheus : Create Prometheus directories] *****
ok: [10.0.1.249] => (item=/opt/monitoring)
ok: [10.0.1.249] => (item=/opt/monitoring/prometheus)
changed: [10.0.1.249] => (item=/opt/monitoring/prometheus/data)

TASK [prometheus : Deploy Prometheus config] *****
changed: [10.0.1.249]

```

FIGURE 5.1 – Execution du playbook Ansible

Chapitre 6

CI/CD — GitHub Actions

6.1 Présentation

Le code source est hébergé sur le dépôt GitHub [Michael-Rousseau/devSecOps](https://github.com/Michael-Rousseau/devSecOps)¹. Quatre workflows GitHub Actions automatisent l'ensemble du cycle de vie :

Workflow	Declencheur	Type	Actions
ci.yml	PR vers main	CI	Lint, test, build (frontend + backend)
cd-frontend.yml	Push sur main	CD	Build Angular, sync S3 (deus-dashboard-frontend-dev)
cd-backend.yml	Push sur main	CD	Build Docker, push ECR, deploy ECS
terraform.yml	PR / Push	IaC	Plan sur PR, Apply sur merge

TABLE 6.1 – Workflows GitHub Actions

6.2 Pipeline CI

Le workflow CI s'exécute sur chaque Pull Request :

```
1 frontend:
2   npm ci -> npm run lint -> npm run build
3
4 backend:
5   cd backend && npm ci -> npm run build
```

Listing 6.1 – Jobs CI en parallèle

La concurrence est gérée pour annuler les exécutions obsolètes sur une même branche.

1. <https://github.com/Michael-Rousseau/devSecOps>

6.3 Deploiement Frontend

1. Build de l'application Angular en mode production
2. Synchronisation des fichiers vers le bucket S3 `deus-dashboard-frontend-dev` via `aws s3 sync`

Note : L'etape d'invalidation CloudFront initialement prevue a ete supprimee car l'hebergement utilise le mode S3 Static Website Hosting (cf. chapitre 2 — Adaptations Learner Lab).

6.4 Deploiement Backend

1. Build de l'image Docker
2. Push vers Amazon ECR `459624991030.dkr.ecr.us-east-1.amazonaws.com/deus-dashboard-backend` (tag : SHA du commit + latest)
3. Mise a jour de la Task Definition ECS sur le cluster `deus-dashboard-dev`
4. Deploiement du service ECS `deus-dashboard-backend` avec attente de stabilite

6.5 Terraform en CI

Sur Pull Request `terraform plan` est execute et le resultat est poste en commentaire sur la PR pour revue.

Sur merge vers main `terraform apply -auto-approve` est execute automatiquement.

6.6 Gestion des credentials AWS Learner Lab

Les credentials AWS Learner Lab (compte 459624991030, region `us-east-1`) expirent toutes les ~4 heures. Un script `scripts/update-aws-keys.sh` permet de mettre a jour simultanement :

- `~/.aws/credentials` (CLI locale)
- Les GitHub Secrets du depot `Michael-Rousseau/devSecOps` (via `gh secret set`)

Les trois secrets concernes sont :

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_SESSION_TOKEN`

Limitation : L'absence de federation OIDC (non autorisée par Learner Lab) impose l'utilisation de clés statiques dans les GitHub Secrets. Ces clés doivent être renouvelées manuellement avant chaque session de déploiement. En production, on privilégierait un provider OIDC (`aws-actions/configure-aws-credentials`) pour éviter le stockage de clés longue durée.

```

it commit -m "ci:
trigger terraform apply" && git push

Linting "DEUS_Dashboard"...

All files pass linting.

[main 2573614] ci: trigger terraform apply
1 file changed, 1 insertion(+)
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 11 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 519 bytes | 519.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github-second:Michael-Rousseau/devSecOps.git
211df01..2573614 main -> main
-> devSecOps git:(main) cd /Users/michael.rousseau/Documents/EPITA/ing2/devSecOps && gh run watch
? Select a workflow run * ci: Terraform [main] 1s ago
X main Terraform - 26359634145
Triggered via push less than a minute ago

JOBS
X Terraform Apply in 4s (ID 77592642244)
  ✓ Set up job
  ✓ Run actions/checkout@v4
  X Run aws-actions/configure-aws-credentials@v4
  - Run hashicorp/setup-terraform@v3
  - Terraform Init
  - Terraform Apply
  ✓ Post Run aws-actions/configure-aws-credentials@v4
  ✓ Post Run actions/checkout@v4
  ✓ Complete job
  - Terraform Plan in 0s (ID 77592642542)

ANNOTATIONS
! Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, aws-actions/configure-aws-credentials@v4. Actions will be forced to run with Node.js 24 by default starting June 2nd, 2026. Node.js 20 will be removed from the runner on September 16th, 2026. Please check if updated versions of these actions are available that support Node.js 24. To opt into Node.js 24 now, set the FORCE_JAVASCRIPT_ACTIONS_TO_NODE24=true environment variable on the runner or in your workflow file. Once Node.js 24 becomes the default, you can temporarily opt out by setting ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
Terraform Apply: .github#2

X The security token included in the request is invalid
Terraform Apply: .github#13

X Run Terraform (26359634145) completed with 'failure'
13s -> devSecOps git:(main)

```

FIGURE 6.1 – Pipeline CI sur une Pull Request

Chapitre 7

Monitoring et Logging

7.1 Présentation

Le monitoring repose sur trois piliers complementaires :

- **CloudWatch** — Monitoring AWS natif (logs, metriques, alarmes)
- **Prometheus + Grafana** — Metriques applicatives et dashboards personnalisés
- **SonarQube** — Qualite du code et analyse statique

7.2 CloudWatch

7.2.1 Logs

Les conteneurs ECS emettent leurs logs via le driver `awslogs` vers le groupe `/ecs/deus-dashboard-b` avec une retention de 14 jours. L'agent CloudWatch (installe par Ansible) collecte egale-ment les logs de l'agent ECS.

7.2.2 Alarmes

Alarme	Condition	Action
ECS CPU elevee	CPU > 80% pendant 10 min	Email SNS
ECS memoire elevee	Memoire > 80% pendant 10 min	Email SNS
ALB erreurs 5xx	> 10 erreurs en 5 min	Email SNS
RDS CPU elevee	CPU > 80% pendant 10 min	Email SNS
RDS stockage bas	< 2 Go restants	Email SNS

TABLE 7.1 – Alarmes CloudWatch configurees

7.3 Prometheus

Prometheus est deploye sur une instance EC2 dediee au monitoring (34.207.159.92), accessible a l'adresse `http://34.207.159.92:9090`. Il collecte les metriques via le protocole pull (scrapping) :

Job	Cible	Metriques
deus-backend	ALB :80/api/metrics	Requetes HTTP (duree, compteur), cache hits/miss, heap Node.js
node-exporter	localhost :9100	CPU, memoire, disque, reseau de l'instance
prometheus	localhost :9090	Metriques internes de Prometheus

TABLE 7.2 – Jobs de scrapping Prometheus

7.3.1 Metriques applicatives

Le backend expose un endpoint `/api/metrics` au format Prometheus grace a la librairie `prom-client` :

```

1 # Duree des requetes HTTP (histogramme)
2 http_request_duration_seconds_bucket{method,route,status_code}
3
4 # Compteur total de requetes
5 http_requests_total{method,route,status_code}
6
7 # Hits et miss du cache DynamoDB
8 cache_hit_total{route}
9 cache_miss_total{route}
10
11 # Metriques Node.js par default
12 nodejs_heap_size_used_bytes
13 nodejs_active_handles_total
14 process_cpu_user_seconds_total

```

Listing 7.1 – Metriques exposees par le backend

7.4 Grafana

Grafana est deploye sur la meme instance que Prometheus (`http://34.207.159.92:3000`) et fournit des dashboards interactifs.

7.4.1 Sources de donnees

- **Prometheus** — Metriques applicatives et systeme
- **CloudWatch** — Metriques AWS (ECS, ALB, RDS)

7.4.2 Dashboard Backend Overview

Le dashboard pre-provisionne comprend :

- Taux de requetes par seconde (par methode, route, code de statut)
- Latence des requetes (percentiles p50 et p95)
- Taux de hit du cache DynamoDB (jauge)
- Taux d'erreurs 5xx
- Utilisation CPU et memoire de l'instance (via Node Exporter)
- Utilisation du heap Node.js

7.4.3 Acces au monitoring

Service	URL	Port
Grafana	http://34.207.159.92:3000	3000
Prometheus	http://34.207.159.92:9090	9090
Node Exporter	http://34.207.159.92:9100	9100

TABLE 7.3 – URLs d'accès aux services de monitoring

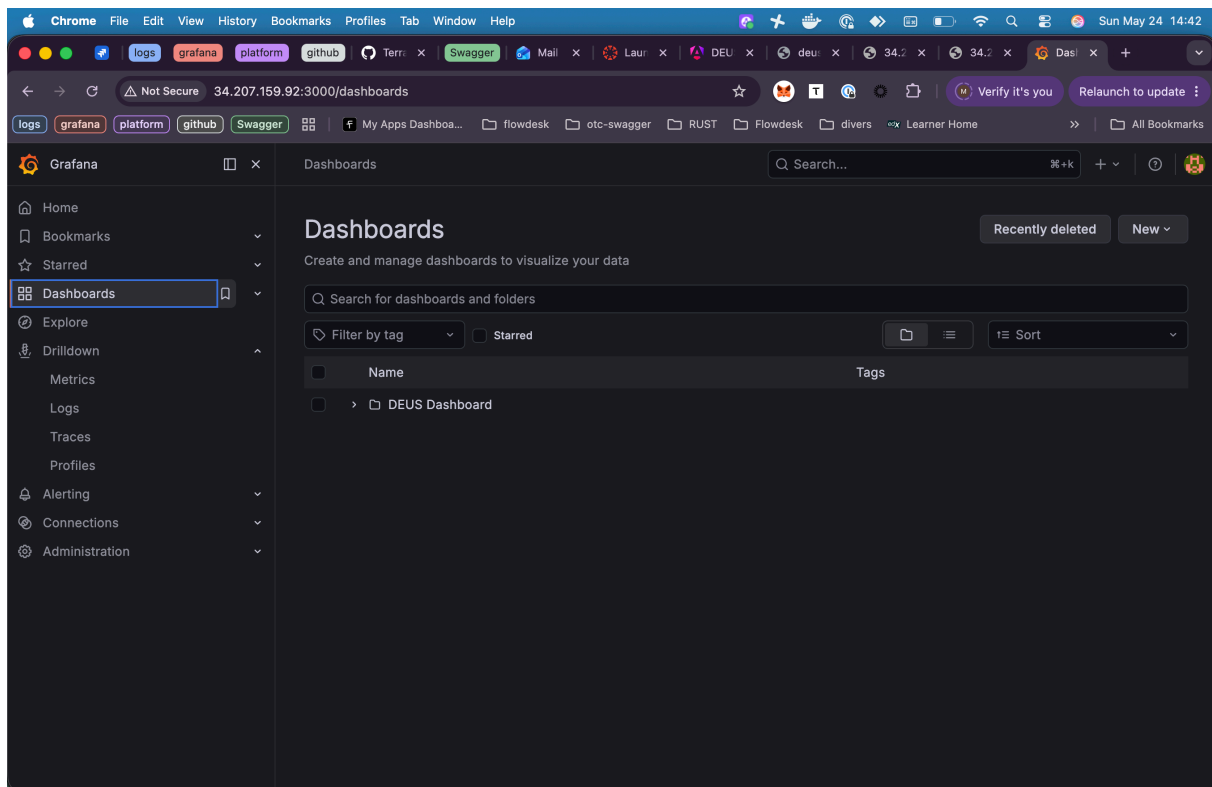


FIGURE 7.1 – Dashboard Grafana — Backend Overview

7.5 SonarQube — Qualite du code

SonarQube analyse le code source pour detecter :

- Les bugs potentiels
- Les vulnerabilites de securite
- Le code duplique
- Les code smells (mauvaises pratiques)
- La couverture de tests

7.5.1 Integration CI

L'analyse SonarQube s'exécute automatiquement dans le pipeline CI (GitHub Actions) apres les etapes de build. Le fichier `sonar-project.properties` configure les sources a analyser :

```
1 sonar.projectKey=deus-dashboard
2 sonar.sources=src,backend/src
3 sonar.exclusions=**/node_modules/**,**/dist/**
```

Listing 7.2 – Configuration SonarQube

7.5.2 Développement local

SonarQube est également disponible en local via Docker Compose sur le port 9000 (<http://localhost:9000>).

7.6 Architecture du monitoring

```
1 Backend (ECS) --scrape--> Prometheus
2               --awslogs--> CloudWatch
3               --sonar-scan--> SonarQube (CI)
4
5 Node Exporter --scrape--> Prometheus
6
7 Prometheus --> Grafana (dashboards)
8 CloudWatch --> Grafana (datasource)
9 CloudWatch --> SNS --> Email (alarmes)
```

Listing 7.3 – Flux de monitoring

Chapitre 8

Securite

8.1 Gestion des secrets — SSM Parameter Store

Les secrets sont stockes dans AWS Systems Manager Parameter Store en tant que `SecureString` (chiffres avec KMS) :

Parametre	Usage	Type
/deus-dashboard/dev/nasa-api-key	Cle API NASA	SecureString
/deus-dashboard/dev/pg-host	Adresse RDS	String
/deus-dashboard/dev/pg-password	Mot de passe DB	SecureString
/deus-dashboard/dev/dynamodb-table	Nom de la table	String

TABLE 8.1 – Parametres SSM

Le role d'execution ECS lit ces parametres au demarrage du conteneur et les injecte comme variables d'environnement.

8.2 IAM — Adaptations Learner Lab

8.2.1 Architecture cible vs. realite Learner Lab

L'architecture ideale prevoyait trois roles IAM distincts suivant le principe du moindre privilege :

Execution Role Permettrait a ECS de : tirer des images ECR, lire les parametres SSM, ecrire dans CloudWatch Logs.

Task Role Permettrait au conteneur de : acceder a DynamoDB (lecture/ecriture sur la table de cache).

Instance Role Permettrait aux instances EC2 de : s'enregistrer au cluster ECS, utiliser SSM Session Manager, envoyer des metriques CloudWatch.

8.2.2 Contrainte Learner Lab

L'environnement AWS Academy Learner Lab interdit la creation de roles IAM (`iam:CreateRole` non autorise). En consequence, l'ensemble des ressources utilisent le role pre-existant **LabRole**, qui dispose de permissions larges couvrant les services necessaires (ECS, ECR, SSM, CloudWatch, DynamoDB, S3, RDS).

```

1 # Au lieu de creer des roles personnalisés
2 data "aws_iam_role" "lab_role" {
3   name = "LabRole"
4 }
5
6 # Execution role et task role ECS
7 execution_role_arn = data.aws_iam_role.lab_role.arn
8 task_role_arn      = data.aws_iam_role.lab_role.arn
9
10 # Instance profile pour les EC2 (ECS, bastion)
11 data "aws_iam_instance_profile" "lab_profile" {
12   name = "LabInstanceProfile"
13 }

```

Listing 8.1 – Utilisation du LabRole dans Terraform

Compromis de securite : Le LabRole dispose de permissions plus larges que necessaire, ce qui contrevient au principe du moindre privilege. En environnement de production, on creerait des roles specifiques avec des politiques restrictives pour chaque composant.

8.3 Reseau — Segmentation VPC

Security Group	Regles entrantes	Cible
sg-alb	80 depuis 0.0.0.0/0	ALB
sg-ecs	32768-65535 depuis sg-alb, 22 depuis sg-bastion	Instances ECS
sg-rds	5432 depuis sg-ecs et sg-bastion	PostgreSQL
sg-bastion	22 depuis IP autorisee	Bastion
sg-monitoring	3000/9090 depuis IP autorisee, 22 depuis sg-bastion	Prometheus/Grafana

TABLE 8.2 – Security Groups et regles

Moindre privilege par references de Security Groups : chaque flux interne est autorise par *reference de Security Group* et non par plage CIDR. Initialement, la base RDS

acceptait le port 5432 depuis l'ensemble des sous-reseaux prives ; la regle a ete restreinte aux seuls `sg-ecs` et `sg-bastion`. Ainsi, meme une ressource compromise placee dans un sous-reseau prive ne peut pas atteindre la base si elle ne porte pas le bon Security Group. De meme, le port 443 de l'ALB, sans listener HTTPS associe (certificat ACM indisponible en Learner Lab), a ete supprime : aucune regle ne reste ouverte sans service derriere.

Les flux autorises se resument a :

```

1 Internet --80--> sg-alb --ports dynamiques--> sg-ecs --5432--> sg-rds
2 Admin    --22--> sg-bastion --22--> sg-ecs / sg-monitoring
3          sg-bastion --5432--> sg-rds    (administration DB)
4 Admin    --3000/9090--> sg-monitoring    (UI Grafana/Prometheus)

```

Listing 8.2 – Flux reseau (moindre privilege)

Seul l'ALB est expose sur Internet. Les instances ECS et la base de donnees sont dans des sous-reseaux prives, accessibles uniquement via l'ALB ou le bastion SSH.

8.4 Bastion SSH

Le bastion est le point d'entree unique pour l'acces SSH aux ressources privees. Il est configure avec :

- Acces SSH restreint a l'IP de l'administrateur
- Instance `t2.micro` (Free Tier)
- Durcissement via Ansible (pas de root SSH, pas d'authentification par mot de passe)

8.5 Cle API NASA

La cle API NASA, auparavant codee en dur dans le code source Angular (visible dans le navigateur), est desormais :

1. Stockee dans SSM Parameter Store (chiffree)
2. Injectee dans le conteneur backend via la Task Definition ECS
3. Utilisee cote serveur uniquement — le frontend ne la voit plus

Chapitre 9

Procédure de déploiement

9.1 Prerequis

- Compte AWS Learner Lab actif
- CLI AWS, Terraform, Ansible et Docker installés localement
- Repository GitHub avec les secrets configurés
- Paire de clés SSH (key pair) créée dans la console AWS

9.2 Etape 1 — Initialisation des credentials

```
1 # Executer le script de rotation
2 ./scripts/update-aws-keys.sh
3
4 # Verifier l'accès (doit retourner le compte 459624991030)
5 aws sts get-caller-identity
6 # {
7 #   "Account": "459624991030",
8 #   "Arn": "arn:aws:sts::459624991030:assumed-role/...",
9 #   "UserId": "...",
10 # }
```

Listing 9.1 – Rotation des clés Learner Lab

Important : Les credentials Learner Lab (compte 459624991030, région `us-east-1`) expirent toutes les ~4 heures. Le script met à jour `~/.aws/credentials` et les GitHub Secrets du dépôt `Michael-Rousseau/devSecOps`.

9.3 Etape 2 — Bootstrap Terraform

```
1 cd terraform/bootstrap
2 terraform init
3 terraform apply
```

Listing 9.2 – Creation du bucket d'état

Cette étape crée le bucket S3 et la table DynamoDB pour le state Terraform. Elle n'est exécutée qu'une seule fois.

9.4 Etape 3 — Déploiement de l'infrastructure

```
1 cd terraform/environments/dev
2 terraform init
3 terraform plan -var="db_password=<MOT_DE_PASSE>" \
4                 -var="nasa_api_key=<CLE_NASA>" \
5                 -var="ssh_key_name=<NOM_KEY_PAIR>"
6 terraform apply
```

Listing 9.3 – Déploiement complet

9.5 Etape 4 — Configuration Ansible

```
1 cd ansible
2 # Mettre à jour l'IP du bastion (35.173.122.117)
3 # dans la config SSH et l'inventaire
4 ansible-playbook playbooks/site.yml
```

Listing 9.4 – Exécution d'Ansible

9.6 Etape 5 — Premier déploiement applicatif

```
1 # Backend --- Build et push vers ECR
2 cd backend
3 aws ecr get-login-password --region us-east-1 | docker login \
4   --username AWS --password-stdin \
5   459624991030.dkr.ecr.us-east-1.amazonaws.com
6
7 docker build -t \
8   459624991030.dkr.ecr.us-east-1.amazonaws.com/deus-dashboard-backend:
9   latest .
9 docker push \
10  459624991030.dkr.ecr.us-east-1.amazonaws.com/deus-dashboard-backend:
    latest
```

```
11
12 # Forcer le redéploiement du service ECS
13 aws ecs update-service --cluster deus-dashboard-dev \
14   --service deus-dashboard-backend --force-new-deployment \
15   --region us-east-1
16
17 # Frontend --- Build et déploiement S3
18 cd ..
19 npm run build
20 aws s3 sync dist/DEUS_Dashboard/browser/ \
21   s3://deus-dashboard-frontend-dev --delete
```

Listing 9.5 – Build et push de l’image Docker

9.7 Etape 6 — Verification

1. Accéder au frontend S3 :
`http://deus-dashboard-frontend-dev.s3-website-us-east-1.amazonaws.com`
L’application Angular doit s’afficher.
2. Vérifier le backend via l’ALB :
`http://deus-dashboard-alb-1071352964.us-east-1.elb.amazonaws.com/api/health`
3. Vérifier que l’APOD se charge (appel via le backend)
4. Tester la liste des événements naturels
5. Consulter le dashboard Grafana : `http://34.207.159.92:3000`
6. Vérifier les métriques Prometheus : `http://34.207.159.92:9090`
7. Vérifier les logs dans le groupe CloudWatch `/ecs/deus-dashboard-backend`

9.8 Ressources deployees

Ressource	Identifiant / URL
Frontend (S3)	deus-dashboard-frontend-dev.s3-website-us-east-1.amazonaws.com
Backend (ALB)	deus-dashboard-alb-1071352964.us-east-1.elb.amazonaws.com
ECR Repository	459624991030.dkr.ecr.us-east-1.amazonaws.com/deus-dashboard-backend
ECS Cluster	deus-dashboard-dev
ECS Service	deus-dashboard-backend
DynamoDB Table	deus-dashboard-cache-dev
S3 Bucket	deus-dashboard-frontend-dev
Bastion IP	35.173.122.117
Monitoring IP	34.207.159.92
Grafana	http://34.207.159.92:3000
Prometheus	http://34.207.159.92:9090
Compte AWS	459624991030
Region	us-east-1

TABLE 9.1 – Recapitulatif des ressources deployees

9.9 Gestion des sessions Learner Lab

Emplacement	Action requise
~/.aws/credentials	Executer <code>update-aws-keys.sh</code>
GitHub Secrets	Repondre “y” dans le script
Backend S3 Terraform	Utilise les memes credentials CLI

TABLE 9.2 – Points de mise a jour des credentials

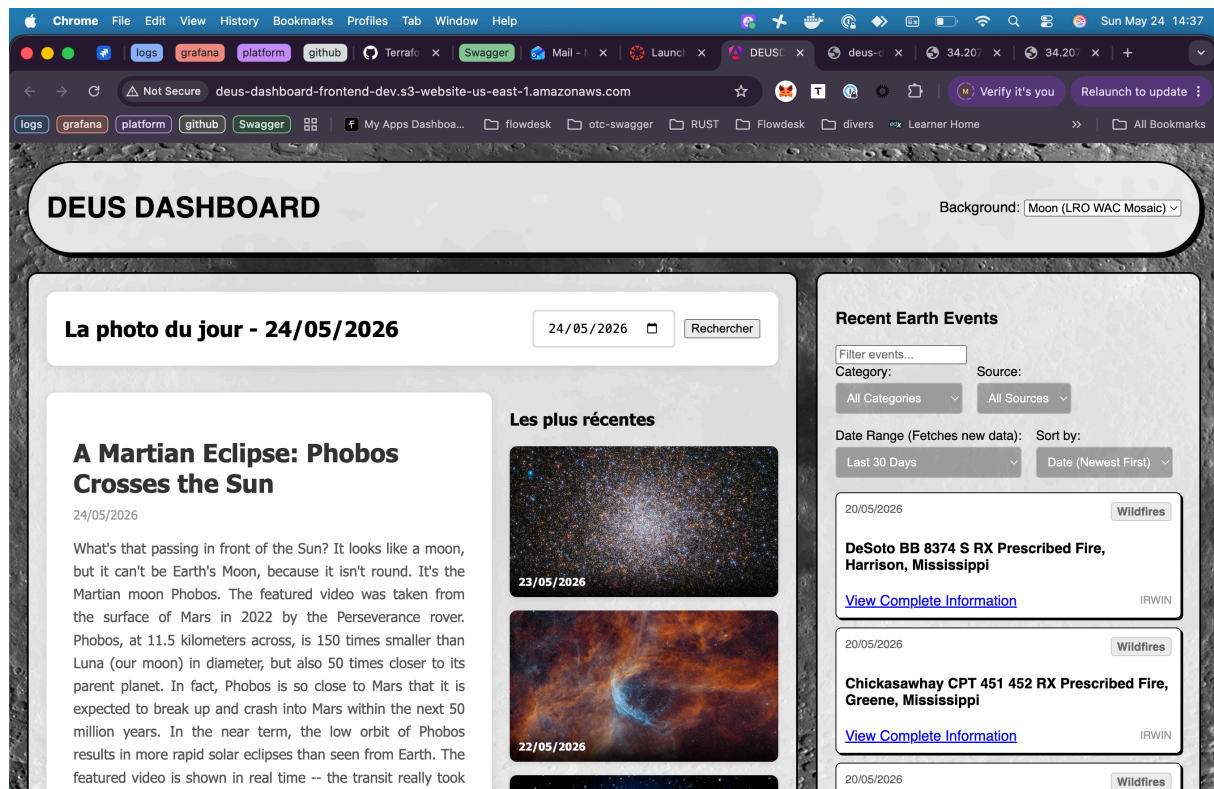


FIGURE 9.1 – Application deployée

Chapitre 10

Conclusion

10.1 Bilan technique

Ce projet a permis de mettre en œuvre une chaîne DevOps complète, couvrant l'ensemble du cycle de vie d'une application web :

- **Conteneurisation** — Docker pour l'isolation et la reproductibilité
- **Infrastructure as Code** — Terraform avec 10 modules couvrant VPC, ECS, RDS, DynamoDB, S3/CloudFront, IAM, SSM, et le monitoring
- **Configuration Management** — Ansible avec 4 rôles pour la configuration automatisée des instances EC2
- **CI/CD** — GitHub Actions avec 4 workflows (CI, déploiement frontend, déploiement backend, Terraform plan/apply)
- **Monitoring** — CloudWatch avec logs, métriques, alarmes et dashboard
- **Sécurité** — IAM least-privilege, VPC segmenté, SSM pour les secrets, bastion SSH

10.2 Difficultés rencontrées

- **Credentials Learner Lab** — L'expiration toutes les 4 heures nécessite une rotation fréquente, résolue par le script automatisé
- **Dépendances circulaires Terraform** — Le module RDS avait besoin du Security Group ECS, qui dépendait de SSM, qui dépendait de RDS. Résolution : utilisation de CIDR blocks au lieu de références de Security Groups
- **Dimensionnement ECS** — Les instances `t3.micro` ne disposent pas de suffisamment de mémoire pour l'agent ECS et les conteneurs. Solution : utilisation de `t2.small`

10.3 Améliorations possibles

- **HTTPS** — Ajout d'un certificat ACM avec un nom de domaine personnalisé
- **Auto Scaling** — Politiques de scaling automatique basées sur la charge
- **Multi-environnement** — Séparation dev/staging/prod avec des workspaces Terraform
- **Authentication** — Intégration d'Amazon Cognito pour l'authentification utilisateur
- **Migration Fargate** — Pour éliminer la gestion des instances EC2 en production
- **Prometheus/Grafana** — Pour un monitoring plus avancé avec des dashboards personnalisés